world use cases without adding unnecessary complexity. Cedar's type system catches common errors early, while its namespace design encourages modular, conflict-free development. By omitting features like regular expressions, loops, and external calls, Cedar keeps policies fast, easy to analyze, and deterministic.

These choices make Cedar approachable to developers, auditable by reviewers, and reliable in production. The language provides enough flexibility to support various authorization styles—from attribute-based decisions to relationship-based access—without becoming fragile or difficult to reason about.

In the next chapter, we will examine how other policy languages address these same challenges. Using PARC as a reference, we will compare Cedar to alternatives like OPA/Rego, OpenFGA, and XACML to better understand the tradeoffs and help you select the appropriate tool for your system.

7.6 Summary

- *Cedar* is a purpose-built policy language designed for real-time, fine-grained authorization decisions using attributes, relationships, and context.
- Policies are organized using three elements: effect (`permit` or `forbid`), scope (principal, action, resource), and conditions (contextual logic).
- Access requests follow the PARC model, and Cedar evaluates them by matching scope and evaluating conditions, using a deny-overrides decision model.
- Cedar offers a rich entity model, allowing policies to reference relationships such as ownership or team membership through first-class entity types and attributes.
- Namespaces and type safety help prevent errors and promote modular policy development by clearly organizing entities and enforcing strict type rules.
- Cedar emphasizes safety, clarity, and analyzability by using strong typing, simple data access, and a minimal set of operators.